

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm
import seaborn as sns
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler,OneHotEncoder
from sklearn.metrics import confusion_matrix
from sklearn.impute import SimpleImputer
from sklearn.metrics import classification_report
from sklearn.metrics import precision_score, recall_score
```

```
pd.read_csv('logistic regression analysis.csv')
```

	satisfaction	Customer Type	Age	Type of Travel	Class	Flight Distance	Seat comfort	Departure/Arrival time convenient	Food and drink	Gate location	...	Online support	Ease Online booking
0	satisfied	Loyal Customer	65	Personal Travel	Eco	265	0	0	0	2	...	2	
1	satisfied	Loyal Customer	47	Personal Travel	Business	2464	0	0	0	3	...	2	
2	satisfied	Loyal Customer	15	Personal Travel	Eco	2138	0	0	0	3	...	2	
3	satisfied	Loyal Customer	60	Personal Travel	Eco	623	0	0	0	3	...	3	
4	satisfied	Loyal Customer	70	Personal Travel	Eco	354	0	0	0	3	...	4	
...	...	...	...	...	...	...	...	...	...	...	...	...	
129875	satisfied	disloyal Customer	29	Personal Travel	Eco	1731	5	5	5	3	...	2	
129876	dissatisfied	disloyal Customer	63	Personal Travel	Business	2087	2	3	2	4	...	1	
129877	dissatisfied	disloyal Customer	69	Personal Travel	Eco	2320	3	0	3	3	...	2	
129878	dissatisfied	disloyal Customer	66	Personal Travel	Eco	2450	3	2	3	2	...	2	
129879	dissatisfied	disloyal Customer	38	Personal Travel	Eco	4307	3	4	3	3	...	3	

129880 rows x 22 columns

```
df = pd.read_csv ('logistic regression analysis.csv')
df.head()
df.tail()
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 129880 entries, 0 to 129879
Data columns (total 22 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   satisfaction                          129880 non-null  str
1   Customer Type                        129880 non-null  str
2   Age                                  129880 non-null  int64
3   Type of Travel                       129880 non-null  str
4   Class                                129880 non-null  str
5   Flight Distance                      129880 non-null  int64
6   Seat comfort                         129880 non-null  int64
7   Departure/Arrival time convenient    129880 non-null  int64
8   Food and drink                       129880 non-null  int64
9   Gate location                        129880 non-null  int64
```

```
10 Inflight wifi service      129880 non-null int64
11 Inflight entertainment    129880 non-null int64
12 Online support            129880 non-null int64
13 Ease of Online booking    129880 non-null int64
14 On-board service          129880 non-null int64
15 Leg room service          129880 non-null int64
16 Baggage handling          129880 non-null int64
17 Checkin service           129880 non-null int64
18 Cleanliness               129880 non-null int64
19 Online boarding           129880 non-null int64
20 Departure Delay in Minutes 129880 non-null int64
21 Arrival Delay in Minutes  129487 non-null float64
dtypes: float64(1), int64(17), str(4)
memory usage: 21.8 MB
```

```
df.describe()
```

	Age	Flight Distance	Seat comfort	Departure/Arrival time convenient	Food and drink	Gate location	Inflight wifi service	Inflight entertainment
count	129880.000000	129880.000000	129880.000000	129880.000000	129880.000000	129880.000000	129880.000000	129880.000000
mean	39.427957	1981.409055	2.838597	2.990645	2.851994	2.990422	3.249130	3.383477
std	15.119360	1027.115606	1.392983	1.527224	1.443729	1.305970	1.318818	1.346059
min	7.000000	50.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	27.000000	1359.000000	2.000000	2.000000	2.000000	2.000000	2.000000	2.000000
50%	40.000000	1925.000000	3.000000	3.000000	3.000000	3.000000	3.000000	4.000000
75%	51.000000	2544.000000	4.000000	4.000000	4.000000	4.000000	4.000000	4.000000
max	85.000000	6951.000000	5.000000	5.000000	5.000000	5.000000	5.000000	5.000000

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 129880 entries, 0 to 129879
Data columns (total 22 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   satisfaction                          129880 non-null str
1   Customer Type                        129880 non-null str
2   Age                                  129880 non-null int64
3   Type of Travel                       129880 non-null str
4   Class                                129880 non-null str
5   Flight Distance                      129880 non-null int64
6   Seat comfort                         129880 non-null int64
7   Departure/Arrival time convenient    129880 non-null int64
8   Food and drink                      129880 non-null int64
9   Gate location                       129880 non-null int64
10  Inflight wifi service                129880 non-null int64
11  Inflight entertainment               129880 non-null int64
12  Online support                       129880 non-null int64
13  Ease of Online booking               129880 non-null int64
14  On-board service                     129880 non-null int64
15  Leg room service                     129880 non-null int64
16  Baggage handling                     129880 non-null int64
17  Checkin service                      129880 non-null int64
18  Cleanliness                         129880 non-null int64
19  Online boarding                      129880 non-null int64
20  Departure Delay in Minutes           129880 non-null int64
21  Arrival Delay in Minutes              129487 non-null float64
dtypes: float64(1), int64(17), str(4)
memory usage: 21.8 MB
```

```
X = df.drop(columns = ['satisfaction'] )
y = df['satisfaction'].map({'dissatisfied' : 0 , 'satisfied' : 1})
print(X)
X.info()
```

	Customer Type	Age	Type of Travel	Class	Flight Distance \
0	Loyal Customer	65	Personal Travel	Eco	265
1	Loyal Customer	47	Personal Travel	Business	2464
2	Loyal Customer	15	Personal Travel	Eco	2138

3	Loyal Customer	60	Personal Travel	Eco	623
4	Loyal Customer	70	Personal Travel	Eco	354
...	...	...	...	...	...
129875	disloyal Customer	29	Personal Travel	Eco	1731
129876	disloyal Customer	63	Personal Travel	Business	2087
129877	disloyal Customer	69	Personal Travel	Eco	2320
129878	disloyal Customer	66	Personal Travel	Eco	2450
129879	disloyal Customer	38	Personal Travel	Eco	4307

	Seat comfort	Departure/Arrival time convenient	Food and drink	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	
...	...	...	...	
129875	5	5	5	
129876	2	3	2	
129877	3	0	3	
129878	3	2	3	
129879	3	4	3	

	Gate location	Inflight wifi service	...	Online support	\
0	2	2	...	2	
1	3	0	...	2	
2	3	2	...	2	
3	3	3	...	3	
4	3	4	...	4	
...	...	...	...	...	
129875	3	2	...	2	
129876	4	2	...	1	
129877	3	3	...	2	
129878	2	3	...	2	
129879	3	3	...	3	

	Ease of Online booking	On-board service	Leg room service	\
0	3	3	0	
1	3	4	4	
2	2	3	3	
3	1	1	0	
4	2	2	0	
...	...	...	...	
129875	2	3	3	
129876	3	2	3	
129877	4	4	3	
129878	3	3	2	
129879	4	5	5	

	Baggage handling	Checkin service	Cleanliness	Online boarding	\
0	3	5	3	2	
1	4	2	3	2	
2	4	4	4	2	
3	1	4	1	3	
4	2	4	2	5	

```
print(y)
```

```
0      1
1      1
2      1
3      1
4      1
..
129875 1
129876 0
129877 0
129878 0
129879 0
Name: satisfaction, Length: 129880, dtype: int64
```

```
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size = 0.20, random_state = 20, stratify =y)
X_train
X_test

y_train
y_test
```

```
18607    0
96304    1
99904    1
44672    0
15616    1
```

```

..
73408      0
41225      0
16164      1
126909     1
78659      0
Name: satisfaction, Length: 25976, dtype: int64

```

```

Total_samples = len(X_train)
print(Total_samples)

```

```

103904

```

```

Total_samples = len(X_test)
print(Total_samples)

```

```

25976

```

```

print(df.isna().sum())
X_clean = X.dropna()
y_clean = y [X_clean.index]

```

```

satisfaction      0
Customer Type     0
Age               0
Type of Travel    0
Class            0
Flight Distance   0
Seat comfort      0
Departure/Arrival time convenient  0
Food and drink    0
Gate location     0
Inflight wifi service  0
Inflight entertainment  0
Online support    0
Ease of Online booking  0
On-board service  0
Leg room service  0
Baggage handling  0
Checkin service   0
Cleanliness       0
Online boarding   0
Departure Delay in Minutes  0
Arrival Delay in Minutes  393
dtype: int64

```

```

Categorical_cols = X_train.select_dtypes(include = ['object' , 'str' , 'category']).columns.tolist()
Numerical_cols = X_train.select_dtypes (include = ['int64' , 'float64' ] ).columns.tolist()

```

```

Categorical_features = Categorical_cols
Numerical_features = Numerical_cols
print('Categorical_features:' ,Categorical_column)
print('Numerical_features:' , Numerical_column)

```

```

Categorical_features: ['Customer Type', 'Type of Travel', 'Class']
Numerical_features: ['Age', 'Flight Distance', 'Seat comfort', 'Departure/Arrival time convenient', 'Food and drink', 'Gate loca

```

```

num_pipeline = Pipeline(steps= [('imputer', SimpleImputer(strategy='median')), ('scaler', StandardScaler())])
cat_pipeline = Pipeline(steps= [('imputer', SimpleImputer(strategy='most_frequent')),('onehot', OneHotEncoder(handle_unknown = '

```

```

df_encoded = pd.get_dummies(df, columns=Categorical_features, drop_first=True)

```

```

preprocessor = ColumnTransformer(transformers = [('num', num_pipeline, Numerical_features,), ('cat',cat_pipeline, Categorical_f

```

```

pipeline = Pipeline(steps=[('preprocessor', preprocessor),
    ('classifier', LogisticRegression(max_iter=1000))])

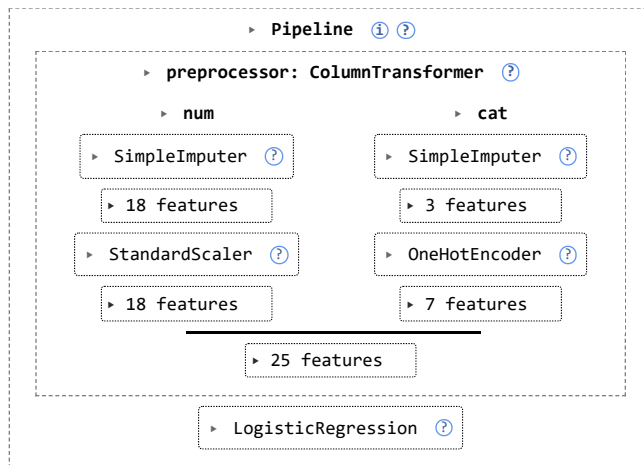
```

```

X_train_clean = preprocessor.fit_transform(X_train)
X_test_clean =preprocessor.transform(X_test)

```

```
pipeline.fit(X_train,y_train)
```



```
y_pred = pipeline.predict(X_test)
print(classification_report (y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.82	0.81	0.81	11759
1	0.84	0.85	0.85	14217
accuracy			0.83	25976
macro avg	0.83	0.83	0.83	25976
weighted avg	0.83	0.83	0.83	25976

```
encoded_cat_features = pipeline.named_steps['preprocessor']\
    .named_transformers_['cat']\
    .get_feature_names_out(Categorical_features).tolist()
all_features = Numerical_features + encoded_cat_features

coefficients = pipeline.named_steps['classifier'].coef_[0]

feature_importance = pd.DataFrame({'Feature': all_features, 'Importance': coefficients})
print(feature_importance.sort_values(by='Importance', ascending=False))
```

	Feature	Importance
7	Inflight entertainment	0.966089
18	Customer Type_Loyal Customer	0.824456
22	Class_Business	0.421840
2	Seat comfort	0.394032
10	On-board service	0.388357
13	Checkin service	0.367253
9	Ease of Online booking	0.325224
11	Leg room service	0.304684
20	Type of Travel_Business travel	0.267905
15	Online boarding	0.195603
5	Gate location	0.159722
8	Online support	0.138644
12	Baggage handling	0.102934
16	Departure Delay in Minutes	0.089182
14	Cleanliness	0.074831
6	Inflight wifi service	-0.118876
0	Age	-0.131888
1	Flight Distance	-0.185674
17	Arrival Delay in Minutes	-0.273862
23	Class_Eco	-0.286985
4	Food and drink	-0.300886
3	Departure/Arrival time convenient	-0.331604
24	Class_Eco Plus	-0.353061
21	Type of Travel_Personal Travel	-0.486110
19	Customer Type_disloyal Customer	-1.042662

```
cm = confusion_matrix(y_test, y_pred)
print("--- CONFUSION MATRIX ---")
```

```
print(f"True Negative (Correctly caught Dissatisfied): {cm[0][0]}")
print(f"False Positive (Wrongly guessed Satisfied):      {cm[0][1]}")
print(f"False Negative (Wrongly guessed Dissatisfied): {cm[1][0]}")
print(f"True Positive (Correctly caught Satisfied):      {cm[1][1]}\n")
```

```
--- CONFUSION MATRIX ---
True Negative (Correctly caught Dissatisfied): 9493
False Positive (Wrongly guessed Satisfied):     2266
False Negative (Wrongly guessed Dissatisfied): 2149
True Positive (Correctly caught Satisfied):     12068
```

```
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
print(f"Precision: {precision:.4f}")
print(f"Recall:      {recall:.4f}\n")
```

```
Precision: 0.8419
Recall:    0.8488
```

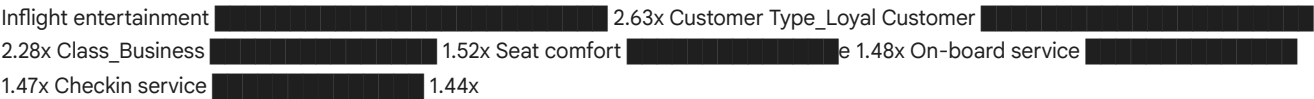
```
print("--- FEATURE IMPORTANCE & ODDS RATIOS ---")
print(f"{'Feature Name':<40} | {'Coefficient':<12} | {'Odds Ratio':<10}")
print("-" * 70)
for feat, coef in zip(all_features, coefficients):
    odds_number = np.exp(coef)

    # Print the row using the calculated odds_number
    print(f"{'feat':<40} | {'coef':<12.4f} | {'odds_number':<10.4f}")
```

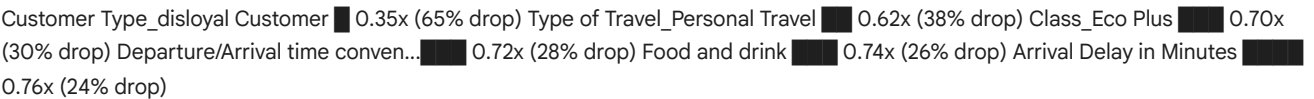
--- FEATURE IMPORTANCE & ODDS RATIOS ---		
Feature Name	Coefficient	Odds Ratio

Age	-0.1319	0.8764
Flight Distance	-0.1857	0.8305
Seat comfort	0.3940	1.4829
Departure/Arrival time convenient	-0.3316	0.7178
Food and drink	-0.3009	0.7402
Gate location	0.1597	1.1732
Inflight wifi service	-0.1189	0.8879
Inflight entertainment	0.9661	2.6276
Online support	0.1386	1.1487
Ease of Online booking	0.3252	1.3843
On-board service	0.3884	1.4746
Leg room service	0.3047	1.3562
Baggage handling	0.1029	1.1084
Checkin service	0.3673	1.4438
Cleanliness	0.0748	1.0777
Online boarding	0.1956	1.2160
Departure Delay in Minutes	0.0892	1.0933
Arrival Delay in Minutes	-0.2739	0.7604
Customer Type_Loyal Customer	0.8245	2.2806
Customer Type_disloyal Customer	-1.0427	0.3525
Type of Travel_Business travel	0.2679	1.3072
Type of Travel_Personal Travel	-0.4861	0.6150
Class_Business	0.4218	1.5248
Class_Eco	-0.2870	0.7505
Class_Eco Plus	-0.3531	0.7025

POSITIVE DRIVERS



NEGATIVE FRICTION



Start coding or [generate](#) with AI.